

relevant-when: Open this drawer when extracting Oracle query results into pandas with python-oracledb Thin mode.

Oracle to Pandas ETL

Use modern python-oracledb in Thin mode. Do not call `oracledb.init_oracle_client()` unless Thick mode is explicitly required.

Connection Handling

Keep credentials outside source files and prefer bind variables in every query.

```
from collections.abc import Iterator, Mapping, Sequence
from datetime import date, datetime
from decimal import Decimal
import oracledb
import pandas as pd

def connect_oracle(user: str, password: str, dsn: str) -> oracledb.Connection:
    return oracledb.connect(user=user, password=password, dsn=dsn)

def create_pool(user: str, password: str, dsn: str) -> oracledb.ConnectionPool:
    return oracledb.create_pool(
        user=user, password=password, dsn=dsn, min=1, max=4, increment=1,
        getmode=oracledb.POOL_GETMODE_WAIT,
    )
```

Use `host:port/service_name` or a configured alias as the DSN. Keep service names, wallets, and secrets in environment or deployment configuration.

SELECT Execution

Keep query execution separate from DataFrame normalization so callers can test each layer.

```
def execute_select(
    conn: oracledb.Connection,
    sql: str,
    params: Mapping[str, object] | None = None,
) -> tuple[list[str], list[tuple[object, ...]]]:
    with conn.cursor() as cur:
        cur.execute(sql, params or {})
        columns = [item[0].lower() for item in cur.description]
        rows = cur.fetchall()
    return columns, rows

sql = """
select account_id, report_date, balance_amt
from source_table
where report_date >= :start_date
"""

with connect_oracle(user, password, dsn) as conn:
    columns, rows = execute_select(conn, sql, {"start_date": date(2026, 1, 1)})
```

Cursor Fetching

Use `arraysize` to reduce round trips for large result sets. Use `fetchmany()` when a result can grow beyond comfortable memory limits.

```
def fetch_batches(
    cur: oracledb.Cursor,
    batch_size: int = 25_000,
) -> Iterator[Sequence[tuple[object, ...]]]:
    cur.arraysize = batch_size
    while rows := cur.fetchmany(batch_size):
        yield rows
```

DataFrame Mapping and Chunks

Build DataFrames from explicit column names, then normalize values in named columns. Yield chunks when extracts can exceed comfortable memory limits.

```
def rows_to_dataframe(
    columns: Sequence[str],
    rows: Sequence[tuple[object, ...]],
) -> pd.DataFrame:
    return pd.DataFrame.from_records(rows, columns=list(columns))
```

```
def read_dataframe(
    conn: oracledb.Connection,
    sql: str,
    params: Mapping[str, object] | None = None,
) -> pd.DataFrame:
    columns, rows = execute_select(conn, sql, params)
    return normalize_db_values(rows_to_dataframe(columns, rows))
```

```
def read_dataframe_chunks(
    conn: oracledb.Connection,
    sql: str,
    params: Mapping[str, object] | None = None,
    batch_size: int = 25_000,
) -> Iterator[pd.DataFrame]:
    with conn.cursor() as cur:
        cur.execute(sql, params or {})
        cur.arraysize = batch_size
        columns = [item[0].lower() for item in cur.description]
        for rows in fetch_batches(cur, batch_size):
            yield normalize_db_values(rows_to_dataframe(columns, rows))
```

Safe Type Conversion

Oracle NUMBER values may arrive as `Decimal`; dates may be `date` or `datetime`; nulls may be `None`. Preserve missing values with pandas nullable dtypes.

```
def normalize_db_values(df: pd.DataFrame) -> pd.DataFrame:
    df = df.replace({None: pd.NA})
    for name in df.columns:
        series = df[name]
        if series.map(lambda value: isinstance(value, Decimal)).any():
            df[name] = series.map(lambda value: float(value) if value is not pd.NA else pd.NA)
        if series.map(lambda value: isinstance(value, (date, datetime))).any():
            df[name] = pd.to_datetime(series, errors="coerce")
    return df
```

```

def convert_columns(
    df: pd.DataFrame,
    integer_cols: Sequence[str] = (),
    numeric_cols: Sequence[str] = (),
    date_cols: Sequence[str] = (),
) -> pd.DataFrame:
    for name in integer_cols:
        df[name] = pd.to_numeric(df[name], errors="coerce").astype("Int64")
    for name in numeric_cols:
        df[name] = pd.to_numeric(df[name], errors="coerce")
    for name in date_cols:
        df[name] = pd.to_datetime(df[name], errors="coerce")
    return df

total_rows = 0
with pool.acquire() as conn:
    for chunk in read_dataframe_chunks(conn, sql, {"start_date": date(2026, 1, 1)}):
        total_rows += len(chunk)

```